SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1402 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I KAVINMITHA.S/192511285 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
kavinmitha .S

Annexure A

EXPERIMENTS

Q1.You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

(i) Accept an arithmetic expression with operators +, -, *, / and parentheses

(ii) Use syntax-directed evaluation to compute values based on operator precedence

(iii) Optimize evaluation by reducing unnecessary temporary computations

(iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

E → E + T { E.val = E1.val + T.val }
E → E – T { E.val = E1.val – T.val }
E → T { E.val = T.val }

T → T * F { T.val = T1.val * F.val }
T → T / F { T.val = T1.val / F.val }
T → F { T.val = F.val }

F → (E) { F.val = E.val }
F → digit { F.val = digit }

ANSWER:

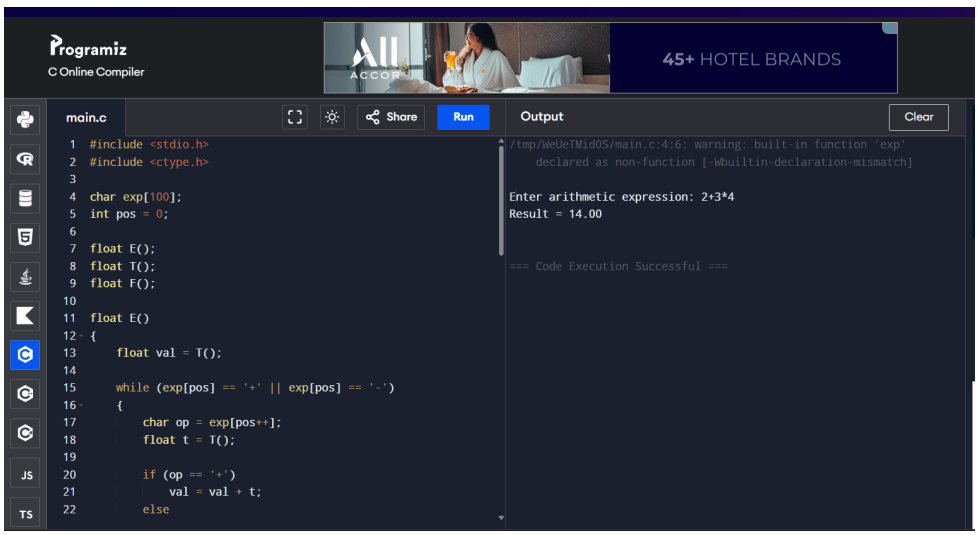
AIM

To implement a C program using **Syntax-Directed Definition (SDD)** with synthesized attributes to evaluate arithmetic expressions containing **+, -, *, /,** and parentheses while maintaining operator precedence.

ALGORITHM

1. Start the program.
2. Read an arithmetic expression from the user.
3. Parse the expression according to the grammar:
    - $E \rightarrow E + T$
    - $E \rightarrow E - T$
    - $E \rightarrow T$
    - $T \rightarrow T * F$
    - $T \rightarrow T / F$
    - $T \rightarrow F$
    - $F \rightarrow (E)$
    - $F \rightarrow digit$
4. Assign synthesized attribute val to every expression.
5. Evaluate multiplication and division before addition and subtraction.
6. Evaluate expressions inside parentheses first.
7. Propagate the computed value upward using synthesized attributes.
8. Display the final value.
9. Stop.

CODE OUTPUT:



Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

(i) Evaluates a postfix expression

(ii) Demonstrates S-attributed definition evaluation

(iii) Shows how intermediate results are computed

ANSWER:

AIM

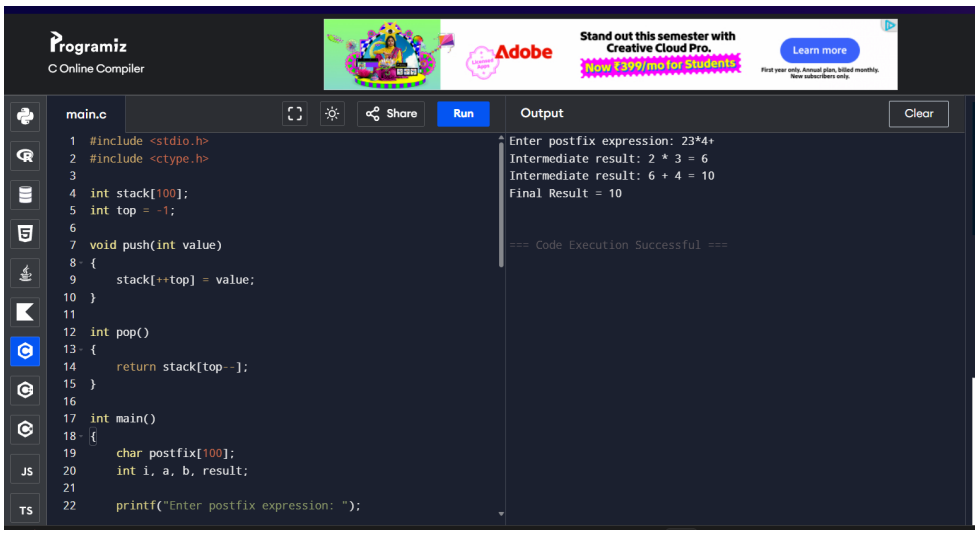
To implement a C program to evaluate a postfix expression using **stack-based bottom-up evaluation** and demonstrate an S-attributed definition.

ALGORITHM

1. Start the program.
2. Read the postfix expression.
3. Initialize an empty stack.
4. Scan the expression from left to right.
5. If the symbol is an operand, push it onto the stack.
6. If the symbol is an operator:
    - Pop the top two operands.
    - Apply the operator.
    - Push the intermediate result onto the stack.
7. Display each intermediate result.
8. After scanning the complete expression, the top of the stack is the final result.
9. Display the final result.

10. Stop.

CODE OUTPUT:



Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

(i).Evaluates a simple expression like `a + b * c`

(ii) Uses function parameters to simulate inherited attributes

(iii). Ensures correct precedence handling

(iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

ANSWER:

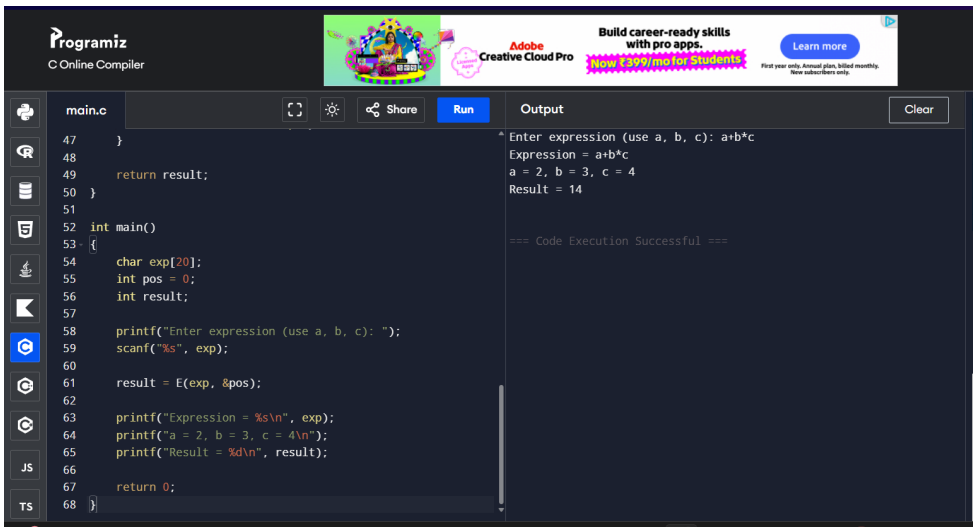
AIM

To implement a C program that demonstrates **L-attributed definition** by using function parameters to simulate inherited attributes and evaluate an expression with correct operator precedence.

ALGORITHM

1. Start the program.
2. Read the expression a+b*c.
3. Use functions to perform top-down evaluation.
4. Pass inherited information through function parameters.
5. Evaluate multiplication before addition.
6. Assign values to identifiers.
7. Pass the inherited value from the parent expression to the child.
8. Compute the final expression value.
9. Display the result.
10. Stop.

CODE OUTPUT:



Q4.In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

(i).Accepts two type expressions (e.g., `int`, `float`, `int*`)

(ii).Compares them for **type equivalence**

(iii).Displays whether they are:

(a).Equivalent

(b).Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

ANSWER:

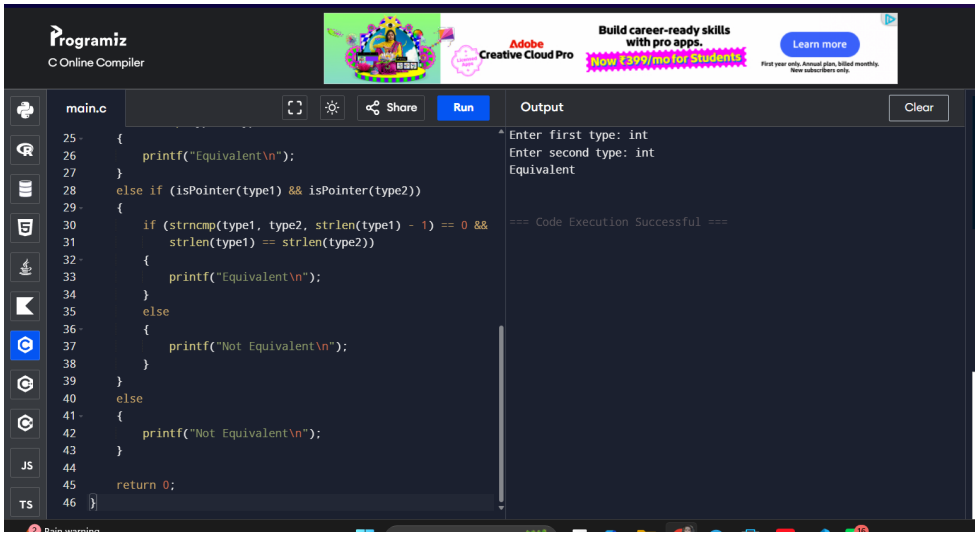
AIM

To implement a C program that compares two type expressions and determines whether they are **equivalent or not equivalent**, including pointer types.

ALGORITHM

1. Start the program.
2. Read the first type expression.
3. Read the second type expression.
4. Compare the two type expressions.
5. If both are exactly the same, they are equivalent.
6. If both are pointer types:
    - Compare their base types.
    - If base types are the same, they are equivalent.
    - Otherwise, they are not equivalent.
7. Display the result.
8. Stop.

CODE OUTPUT:



Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

(i). Accepts two operands and their types

(ii). Checks whether the operation (e.g., `+`, `*`) is valid

(iii). Reports:

(a).Valid expression

(b).Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

ANSWER:

AIM

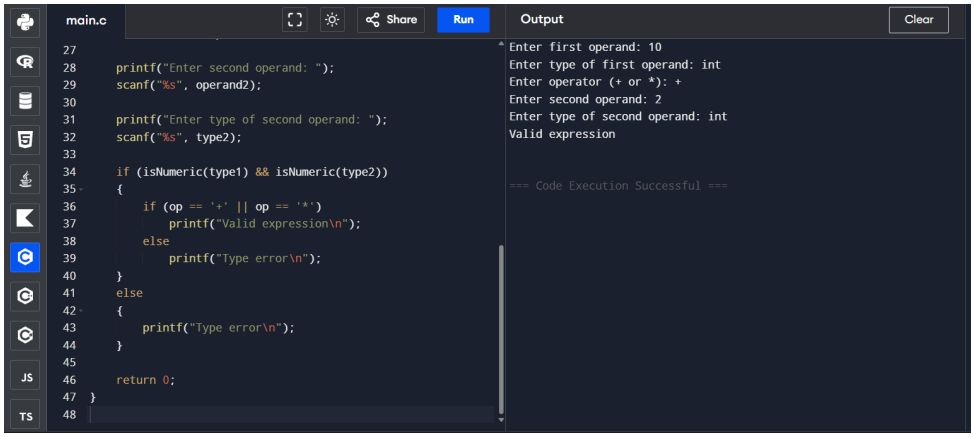
To implement a C program for **semantic type checking** that detects type errors in arithmetic expressions.

ALGORITHM

1. Start the program.
2. Read the first operand and its type.
3. Read the second operand and its type.
4. Read the arithmetic operator.
5. Check the types of both operands.

6. If both operands are numeric types such as int or float, allow the arithmetic operation.
7. If one operand is a pointer type and the other is an incompatible type, report a type error.
8. Display either:
    - Valid expression
    - Type error
9. Stop.

CODE OUTPUT:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation